

end to end :

```
(.venv) PS C:\ecole\Data M2\Sec 1\real-time-sales-devops-tp-main> docker exec sales-postgres psql -U sales -d sales -c "SELECT * FROM processed_orders;"
order_id | customer_id | product_id | quantity | unit_price | total_amount | event_timestamp | processed_at
-----
ORD-6063A5ED5E | C001 | P001 | 2 | 49.90 | 99.80 | 2026-09-29 09:16:39.027078+00 | 2026-09-29 09:28:23.130797+00
ORD-08D7469FFC | C001 | P001 | 2 | 49.90 | 99.80 | 2026-09-29 09:18:44.651178+00 | 2026-09-29 09:28:23.130797+00
(2 rows)
```

12. Jenkinsfile :

The screenshot shows the Jenkins web interface for a build named 'sales-pipeline-test' with ID '#21' (dated 29 sept. 2026, 12:37:12). The build status is 'Success' (green checkmark). The left sidebar contains navigation links: Status, Changes, Console Output, Edit Build Information, Supprimer le build '#21', Git Build Data, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main area displays build details: 'Artefacts du build' (coverage.xml, 2.29 KiB), 'Lancé par un utilisateur anonyme', 'Revision: 78248c02c568be66766d280093bfa9f7d3d66496', and 'Repository: https://github.com/Yac-Leb/Sec1_TP_REALTIME.git'. A step 'Run integration tests with Python 3.11' is shown with a link to the console output. On the right, there are buttons for 'Ajouter une description' and 'Conserver cette construction sans limite de temps', along with a timer indicating the build started 3 minutes and 25 seconds ago and lasted 44 seconds.

13. SonarQube

The screenshot shows the SonarQube 'Add Secret text' dialog box. The background shows the 'Installations de SonarQube' page with fields for 'Nom' (SonarQube), 'URL du serveur' (http://sonarqube:9000), and 'Server authentication token' (- aucun -). The dialog box has a title bar with a back arrow and a close button. It contains the following fields: 'Portée' (Scope) set to 'Global (Jenkins, agents, items, etc...)', 'Secret' (a masked text input), 'ID' set to 'sonar-token', and 'Description' set to 'SonarQube Token'. A blue 'Create' button is at the bottom right.

Jenkins / sales-pipeline-test / #23

#23 (29 sept. 2026, 13:02:07) Ajouter une description Conserver cette construction sans limite de temps

Status

- </> Changes
- Console Output
- Edit Build Information
- Supprimer le build "#23"
- Git Build Data
- Restart from Stage
- Replay

Artefacts du build

- coverage.xml 2.29 KiB [view](#)

Lancé par un utilisateur anonyme

git

Revision: 424aeca1408ff13b843923dac15301f59f5c5e1e
Repository: https://github.com/Yac-Leb/Sec1_TP_REALTIME.git

- refs/remotes/origin/main

</> Run integration tests with Python 3.11 424aeca
 lebkiri.yacinee at 13:01 29/09/2026

Démarrée il y a 1 mn 35 s.
 A duré 54 s

Real-Time Sales DevOps TP Public Passed

Last analysis: 1 minute ago • 63 Lines of Code • Python

Category	Score	Status
Security	0	Passed
Reliability	0	Passed
Maintainability	1	Warning
Hotspots Reviewed	—	Warning
Coverage	14.6%	Warning
Duplications	0.0%	Passed

- Bugs / Reliability : 0
- Vulnerabilities / Security : 0
- Code Smells / Maintainability : 1
- Coverage : 14,6 %
- Duplications : 0,0 %
- Quality Gate : Passed

14. Quality code : *

#27 (29 sept. 2026, 14:30:41) Ajouter une description

Lancé par un utilisateur anonyme

git

Revision: 33e9f7af1f57ad9ad3f0d7bcb0e0f87319b8d131
Repository: https://github.com/Yac-Leb/Sec1_TP_REALTIME.git

- refs/remotes/origin/main

</> Run integration tests with Python 3.11 c6d7243
 lebkiri.yacinee at 14:26 29/09/2026

Run integration tests with Python 3.11 33e9f7a
 lebkiri.yacinee at 14:30 29/09/2026

Un Quality Gate SonarQube a été intégré au pipeline Jenkins afin de contrôler automatiquement la qualité du code après l'analyse SonarQube.

Le pipeline utilise la commande suivante :

```
waitForQualityGate abortPipeline: true
```

Jenkins attend ainsi le résultat du Quality Gate avant de poursuivre. Si les critères de qualité sont respectés, la pipeline est validée ; dans le cas contraire, elle est automatiquement arrêtée en échec.

Lors de notre exécution, SonarQube a retourné :

SUCCESS

15. Amélioration du code :

Bugs / Reliability	: 0
Vulnerabilities	: 0
Code Smells	: 1
Duplications	: 0,0 %
Coverage	: 14,6 %

apres :

 Home  Cours : Atelier ATL-...  MY DIGITAL SCHOO...  Liste candidatures e...  Apprenti Assistant C

[Issues](#) [Rules](#) [Quality profiles](#) [Quality gates](#) [Administration](#) [More](#) 

☐ Bulk Change

No Issues. Hooray!

16. Industrialisation complète :

#29



 #29 (29 sept. 2026, 14:47:51)

 Ajouter une description

Conserver cette construction sans limite de temps



Artefacts du build
 coverage.xml 2.29 KiB [view](#)

Démarrée il y a 3 mn 24 s.
A duré 17 s



Lancé par un utilisateur anonyme



Revision: f9b9b0f8bbfdbf55d6c38950cec2f1e3ff979a00
Repository: https://github.com/Yac-Leb/Sec1_TP_REALTIME.git
• refs/remotes/origin/main



 Test regression pipeline f9b9b0f
|  lebkiri.yacinee at 14:47 29/09/2026

avec assert event["total_amount"] == 999.0

on remet le code initiale :



#31 (29 sept. 2026, 14:56:45)

 Ajouter une description

Conserver cette construction sans limite



Artefacts du build
 coverage.xml 2.29 KiB [view](#)

Démarre
A duré 4 s



Lancé par un utilisateur anonyme



Revision: 9115f072f19375aeed3dc0ba5ba5db38db4d8fed
Repository: https://github.com/Yac-Leb/Sec1_TP_REALTIME.git
• refs/remotes/origin/main



 Test regression pipeline 9115f07
|  lebkiri.yacinee at 14:51 29/09/2026